Laboratório SOsim – Gerência do Processador

Exercício 1. O que acontece se o tempo de time slice (quantum) aumentar ou diminuir?

Observado: com escalonamento circular e dois processos de mesma prioridade, após três minutos o processo CPU-bound acumulou cerca de 160 s de UCP contra aproximadamente 20 s do I/O-bound. O CPU-bound alternava apenas entre Execução e Pronto, saindo do processador somente por fim de fatia de tempo, enquanto o I/O-bound passava para o estado de Espera bem antes de esgotar o seu quantum.

Aumentando o quantum, há menos preempções e menos trocas de contexto, o que reduz o overhead e aumenta o tempo de UCP dedicado a trabalho útil, favorecendo o processo CPU-bound. Em compensação, o tempo de resposta piora: no exemplo acima, o processo I/O-bound passaria a esperar mais tempo na fila de prontos até que o CPU-bound liberasse o processador. No limite, o escalonamento circular degenera em um FIFO não preemptivo.

Diminuindo o quantum, o tempo de resposta melhora e o sistema fica mais interativo, beneficiando o processo I/O-bound. Porém, o número de trocas de contexto cresce e o custo de salvar e restaurar o contexto passa a consumir uma parcela significativa do processador, reduzindo o throughput.

Vale notar que o I/O-bound é pouco afetado de forma direta, já que se bloqueia antes de esgotar o quantum: o parâmetro influencia principalmente o CPU-bound e o tempo de espera na fila de prontos. Existe, portanto, um compromisso entre eficiência e tempo de resposta.

Exercício 2. O que acontece se o tempo de espera do processo I/O-bound aumentar ou diminuir?

Observado: com o I/O-bound em prioridade 4 e o CPU-bound em prioridade 3, sempre que o I/O-bound concluía a operação de E/S ele preemptava imediatamente o CPU-bound, que retornava à fila de prontos sem ter esgotado o quantum. Os tempos de UCP ao final dos três minutos foram próximos aos do Exercício 1 (cerca de 155 s contra 25 s), mas com um número visivelmente maior de trocas de contexto e um tempo de resposta bem menor para o I/O-bound.

Aumentando o tempo de espera de E/S, o processo permanece mais tempo bloqueado e solicita a UCP com menor frequência. O CPU-bound é preemptado menos vezes e amplia ainda mais sua participação no uso do processador, aproximando o comportamento do sistema ao observado no Exercício 1; o overhead de troca de contexto diminui.

Diminuindo o tempo de espera de E/S, o processo retorna à fila de prontos com muito mais frequência e, por ter prioridade maior, preempta o CPU-bound repetidamente. Cresce o número de trocas de contexto e o overhead, e a fatia efetiva de UCP do CPU-bound cai. Em situação extrema, o CPU-bound pode sofrer postergação indefinida (starvation) e o tempo gasto com trocas de contexto pode superar o de processamento útil.

Exercício 3. Quais devem ser os critérios para determinar as prioridades de processos?

Observado: ao inverter as prioridades (CPU-bound com 4 e I/O-bound com 3), o CPU-bound monopolizou o processador. Como ele nunca se bloqueia, saía da UCP apenas por fim de fatia de tempo e, por ser o de maior prioridade na fila de prontos, era imediatamente reescalonado. O I/O-bound ficou praticamente parado no estado Pronto, com tempo de processador próximo de zero – um caso claro de postergação indefinida (starvation), que ainda deixa os dispositivos de E/S ociosos. Esse resultado mostra que a prioridade não pode ser atribuída de forma arbitrária; os

principais critérios são:

- **Perfil do processo:** processos I/O-bound devem ter prioridade maior que os CPU-bound. Como liberam rapidamente a UCP, atendê-los primeiro custa pouco tempo de processador e mantém os dispositivos de E/S ocupados, aumentando o throughput do sistema.
- **Tipo de processamento:** processos interativos e de tempo real, com exigência de tempo de resposta ou prazos (deadlines), devem ter prioridade sobre processos batch.
- **Natureza do processo:** processos do sistema operacional e rotinas do núcleo têm prioridade sobre os processos de usuário.
- **Importância da aplicação e critérios administrativos:** urgência do serviço, perfil do usuário ou departamento e políticas de uso do sistema.
- **Tempo de espera na fila de prontos:** deve haver um mecanismo de envelhecimento (aging), aumentando gradualmente a prioridade de quem espera há muito tempo, justamente para evitar a situação observada neste exercício.
- **Prioridade dinâmica em vez de estática:** a prioridade não deve ser fixada apenas na criação do processo, pois seu comportamento pode mudar durante a execução.

Exercício 4. Qual a vantagem desse escalonamento em processos I/O-bound de perfis diferentes?

Observado: mesmo criando os dois processos com a mesma prioridade inicial, o sistema elevou automaticamente a prioridade do processo que abandonava a UCP por E/S e reduziu a daquele que consumia toda a fatia de tempo. Em pouco tempo o I/O-bound passou a ser escalonado assim que ficava pronto, reproduzindo na prática o resultado do Exercício 2, mas sem nenhuma configuração manual de prioridades.

Os processos I/O-bound não são todos iguais: alguns realizam operações de E/S muito frequentes e curtas, como os que interagem com teclado, terminal ou rede, e outros fazem operações mais espaçadas e demoradas, como acessos a disco ou impressão. Com prioridade estática, como no Exercício 2, todos receberiam o mesmo tratamento e precisariam ser classificados manualmente, um a um.

No mecanismo adaptativo, a prioridade é ajustada pelo próprio sistema conforme o comportamento real de cada processo: quanto maior a fração de tempo que ele passa bloqueado e menor o uso que faz da UCP, maior tende a ser sua prioridade dinâmica. Assim, o processo mais interativo obtém menor tempo de resposta sem prejudicar o throughput, já que consome pouco tempo de processador. O ajuste ainda acompanha mudanças de perfil ao longo da execução e reduz o risco de starvation visto no Exercício 3, tornando o escalonamento mais justo e eficiente em cargas de trabalho heterogêneas.